

```

"""
Orca V2 Backtest - Head-to-Head vs V1
=====

Runs both organisms across the full historical dataset (2020-2026)
covering COVID crash, bull run, 2022 bear, rate hike cycle, 2024 recovery.

Metrics tracked:
- Total return
- Max drawdown
- Win rate
- Profit factor
- Efficiency (return per risk unit)
- Constraints learned (learning progression)
- Survival (did it die before end of test?)
"""

import sys
import os
import pickle
import time

sys.path.insert(0, os.path.dirname(__file__))

from trading.orca_v2 import OrcaV2
from trading.orca import OrcaTrader

def run_v2_backtest(historical_data, starting_capital=10000.0, verbose=True):
    """Run OrcaV2 across historical data"""
    print("\n" + "="*60)
    print("  ORCA V2 BACKTEST - SIGHTED PREDATOR")
    print("="*60)

    orca = OrcaV2(starting_capital=starting_capital, verbose=verbose)

    results = []
    start_time = time.time()

    for day_data in historical_data:
        if not orca.state.is_viable():
            print(f"\n💀 ORGANISM DIED on day {orca.days_active}")
            break

        # Mark market as open for the backtest

```

```

    # (historical data has market_open=False, we force it open)
    day_data_copy = {**day_data, 'market_open': True}

    result = orca.cycle(day_data_copy)
    results.append(result)

elapsed = time.time() - start_time
orca.print_status()

print(f" 🕒 Backtest time: {elapsed:.1f}s for {len(results)} days")

return orca, results

def run_v1_backtest(historical_data, starting_capital=10000.0):
    """Run OrcaV1 across historical data for comparison"""
    print("\n" + "="*60)
    print("  ORCA V1 BACKTEST - BLIND PREDATOR")
    print("="*60)

    orca = OrcaTrader(starting_capital=starting_capital)

    results = []
    days_run = 0

    for day_data in historical_data:
        if not orca.state.is_viable():
            print(f"\n 🧟 V1 DIED on day {days_run}")
            break

        try:
            day_data_copy = {**day_data, 'market_open': True}

            # V1 needs watchlist populated - use all_prices as watchlist
            if not day_data_copy.get('watchlist'):
                watchlist = []
                for sym, price_data in day_data_copy.get('all_prices', {}).items():
                    if sym == 'SPY':
                        continue
                    price = price_data.get('close', 0) if isinstance(price_data, dict) else price_data
                    watchlist.append({
                        'symbol': sym,
                        'price': price,
                        'volume_ratio': 1.2, # neutral
                        'breakout': False,
                        'rsi': 50,
                        'setup_type': 'neutral',

```

```

        })
        day_data_copy['watchlist'] = watchlist
        day_data_copy['watchlist_size'] = len(watchlist)

        result = orca.cycle(day_data_copy)
        results.append(result)
        days_run += 1
    except Exception as e:
        print(f" V1 error day {days_run}: {e}")
        days_run += 1

status = orca.get_status()
portfolio_dollars = status['portfolio_value'] * starting_capital

print(f"\n RESULT:")
print(f" 💰 Portfolio:  ${portfolio_dollars:,.2f} ({status['portfolio_value'] / starting_capital - 1: .1%})")
print(f" 📉 Drawdown:  {status.get('drawdown', 0):.1%}")
print(f" 🎯 Win Rate:   {status.get('win_rate', 0):.1%}")
print(f" 📊 Days:       {days_run}")
print(f" 🧠 Constraints: {status.get('constraints_learned', 0)}")

return orca, results

def compare_results(v1_orca, v2_orca, starting_capital):
    """Print side-by-side comparison"""
    print("\n" + "="*60)
    print(" V1 vs V2 COMPARISON")
    print("="*60)

    v1 = v1_orca.get_status()
    v2 = v2_orca.get_status()

    v1_dollars = v1['portfolio_value'] * starting_capital
    v2_dollars = v2['portfolio_value'] * starting_capital

    metrics = [
        ("Portfolio Value", f"${v1_dollars:,.2f}", f"${v2_dollars:,.2f}"),
        ("Total Return", f"{(v1['portfolio_value']-1)*100:+.1f}%", f"{(v2['portfolio_value']-1)*100:+.1f}%"),
        ("Max Drawdown", f"{v1_orca.state.max_drawdown:.1%}", f"{v2_orca.state.max_drawdown:.1%}"),
        ("Win Rate", f"{v1.get('win_rate',0):.1%}", f"{v2.get('win_rate',0):.1%}"),
        ("Total Trades", str(v1.get('total_trades', 0)), str(v2.get('total_trades', 0))),
        ("Efficiency", "N/A", f"{v2.get('efficiency', 0):.3f}"),
        ("Constraints", str(v1.get('constraints_learned', 0)), str(v2.get('constraints_learned', 0))),
        ("Skills", "N/A", str(v2.get('skills_developed', 0))),
        ("Episodes", "N/A", str(v2.get('total_episodes', 0))),
        ("Survived", "✅" if v1.get('alive') else "💀", "✅" if v2.get('alive') else "💀")
    ]

```

```

]

print(f"  {'Metric':<20} {'V1 (Blind)':<18} {'V2 (Sighted)':<18}")
print(f"  {'-'*56}")
for metric, v1_val, v2_val in metrics:
    print(f"    {metric:<20} {v1_val:<18} {v2_val:<18}")

print(f"\n{'='*60}")

# Winner assessment
v2_won = v2_dollars > v1_dollars
v2_safer = v2_orca.state.max_drawdown < v1_orca.state.max_drawdown

if v2_won and v2_safer:
    print(" 🏆 V2 wins: Better returns AND better risk management")
elif v2_won:
    print(" 📈 V2 wins on returns, but review drawdown")
elif v2_safer:
    print(" 🛡️ V2 wins on risk control – tighter drawdown")
else:
    print(" ⚠️ V1 performed comparably – review V2 signal calibration")
print()

def run_learning_progression_analysis(v2_orca):
    """Show how constraints evolved over time"""
    print("\n" + "="*60)
    print("  LEARNING PROGRESSION ANALYSIS")
    print("="*60)

    constraints = v2_orca.memory.constraints
    skills = v2_orca.memory.skills

    if constraints:
        print(f"\n  Top 5 Strongest Constraints (Learned Avoidances):")
        sorted_c = sorted(constraints, key=lambda c: c.strength(), reverse=True)
        for i, c in enumerate(sorted_c[:5]):
            print(f"    {i+1}. [{c.strength():.2f}] {c.pattern}")
            print(f"          Condition: {c.condition}")
            print(f"          Violations: {c.violation_count} | Reinforcements: {c.r")
    else:
        print("\n  No constraints learned yet – need more trading cycles")
        print("    (Expected after 30+ days with loss events)")

    if skills:
        print(f"\n  Skills Developed ({len(skills)} total):")
        for skill in skills[:5]:

```

```

        print(f"    ✓ {skill.name}: {skill.success_rate:.1%} success rate")
    else:
        print("\n    No skills developed yet")

    print(f"\n    Memory: {len(v2_orca.memory.episodes)} total episodes")
    print(f"    Scars:  {len(v2_orca.memory.scars)} accumulated")

if __name__ == '__main__':
    print("Loading historical data...")
    with open('historical_data.pkl', 'rb') as f:
        data = pickle.load(f)

    print(f"Loaded {len(data)} days ({data[0]['date']} → {data[-1]['date']})")

    CAPITAL = 10000.0
    VERBOSE = True

    # Run V2
    v2_orca, v2_results = run_v2_backtest(data, starting_capital=CAPITAL, verbose

    # Run V1 for comparison (less verbose)
    v1_orca, v1_results = run_v1_backtest(data, starting_capital=CAPITAL)

    # Compare
    compare_results(v1_orca, v2_orca, CAPITAL)

    # Learning analysis
    run_learning_progression_analysis(v2_orca)

```